02

MGU BCA - 3RD SEM

Software Engineering

Module 2 - Requirements, Risk and Planning

By Aben Sojan

Exam - 10 Nov 2026

Complete question bank: every important, repeated and sure-mark question for this module, each with a full-mark model answer. Nothing important left out.

EXAM PATTERN (from real MGU question papers)

SF HONOURS EXAM PATTERN - 50 marks, 1.5 hours. Part A: 10 x 2 marks, all compulsory; Q5-Q7 come from this module (requirements and planning). Part B: 5 x 5 marks, answer any 3 of 5. Part C: 2 x 15 marks, answer any 1 of 2; the model paper essay is requirements elicitation and analysis - the single most likely 15-marker. COCOMO appeared 4 times (May 2019, Oct 2021, Aug 2022, Feb 2025) - sure mark; function points 2x. RMMM, risk tables and agile planning have zero past-paper precedent: learn them from the syllabus as IMPORTANT, not as repeats.

TOPIC 1

Requirements Engineering Basics

1

Requirements engineering is the process of finding out what the software must do.

Functional requirements: what the system does, like login or payment.

Non-functional requirements: how well it does it, like speed, security and usability.

Getting requirements wrong is the most expensive mistake in software.

QUESTION BANK

1 **What is requirements engineering? Why is it important?**

IMPORTANT

Requirements engineering is the process of discovering, documenting and managing what the customer wants the software to do.

It starts by talking to all stakeholders like customers, users and managers to collect their needs.

These needs are written clearly in a requirements document so nothing is missed or misunderstood.

Requirements are then checked and agreed upon before any design or coding begins.

It is the first and most important stage of software development.

Studies show that fixing a wrong requirement after coding costs many times more than fixing it at the start.

Many project failures happen not because of bad coding but because the software was built for the wrong needs.

For example, if a bank app misses the fund transfer feature, the whole product fails no matter how good the code is.

So good requirements engineering saves time, money and customer trust.

2 **Distinguish between functional and non-functional requirements with examples.**

IMPORTANT

Functional requirements describe what the system must do, the services it provides.

For example, an online shop must allow user registration, add to cart and online payment.

Non-functional requirements describe how well the system must do it, the quality limits.

For example, the payment page must load within 2 seconds and use secure encryption.

Functional requirements are tested by checking if the feature works or not.

Non-functional requirements are tested by measuring speed, security, usability and reliability.

An ATM example: dispensing cash is functional, doing it within 30 seconds is non-functional.

Both types are equally important because a feature that works too slowly is as bad as a missing feature.

In the exam, always pair each functional example with a matching non-functional limit to get full marks.

3 **Give four examples of non-functional requirements.**

IMPORTANT

Performance: the system must display search results within 2 seconds.

Security: all user passwords must be stored in encrypted form and never as plain text.

Usability: a new user must be able to learn and use the app without help within 10 minutes.

Reliability: the system must not fail more than once in a month of normal use.

Each of these is written with a clear number so it can be tested.

A vague statement like the system should be fast is not a proper non-functional requirement.

For example, in a railway booking site, non-functional needs decide whether crores of users can book tickets at once.

Good non-functional requirements turn customer wishes into measurable targets for the team.

TOPIC 2

Requirements Document and Specification

2

The requirements document records everything the software must do.

It includes purpose, scope, functional and non-functional requirements, and constraints.

A specification describes each requirement precisely enough to build and test it.

Both customer and developers agree on this document before building.

QUESTION BANK

4 What is a requirements document? Explain its contents. [Asked: Model paper]

REPEATED

The requirements document is the agreed written record of everything the software must do.

It acts as the contract between the customer and the development team.

Its contents begin with the introduction: the purpose of the system and who will use it.

Next comes the system scope: what is included and what is not included.

Then the functional requirements: the full list of services the system must provide.

Then the non-functional requirements: speed, security, usability and reliability targets.

It also lists constraints like budget, deadlines and technology limits, plus the assumptions made.

Both customer and developers sign off on this document before building starts.

Any change later must go through a formal change process because this document is the baseline.

5 What is a software specification? What makes it good?

IMPORTANT

A software specification describes each requirement precisely enough to build and test it.

A good specification must be correct: it says exactly what the customer wants.

It must be unambiguous: each line has only one possible meaning.

It must be complete: nothing important is left out.

It must be verifiable: there is a way to test whether it is done.

For example, the system shall respond within 2 seconds is verifiable, while fast response is not.

It must be consistent: no two requirements should contradict each other.

Bad specifications cause arguments later, so writing them well saves the whole project.

6 What do you mean by scope in SRS?

IMPORTANT

Scope in an SRS, the Software Requirements Specification, defines the boundaries of what the software will and will not do.

It states the main goals and the features included in the project.

It also clearly lists what is excluded, so expectations stay realistic.

A clear scope prevents scope creep, where extra demands keep entering during development.

It helps in estimating cost and time, because the team knows exactly what must be built.

It gives a basis for accepting or rejecting change requests later.

Both the customer and the developers sign off on the scope before work begins.

Example: for a library app, the scope includes book issue and return but excludes online fee payment, which is stated as out of scope.

In short, scope draws the fence around the project so everyone agrees on what is inside.

TOPIC 3

Requirements Engineering Processes

3

Requirements engineering has four main processes.

Elicitation and analysis: gather needs from stakeholders and understand them.

Validation: check that requirements are correct and complete.

Management: control changes to requirements during the project.

These run in a loop until everyone agrees.

QUESTION BANK

7 Explain requirements elicitation and analysis. [Asked: Model paper]

SURE MARK

Requirements elicitation and analysis is the heart of requirements engineering, where the team discovers and understands what to build.

Elicitation means gathering requirements from stakeholders through interviews, questionnaires, observation and workshops.

FAST, the Facilitated Application Specification Technique, brings customers and developers together in a guided joint session.

Prototyping is also used: showing a rough version helps customers say what they really want.

Analysis then studies the gathered requirements for conflicts, gaps, overlaps and feasibility.

Conflicting requirements are negotiated and resolved with the stakeholders.

Unclear requirements are clarified before they are written into the requirements document.

The output is a clear, agreed and feasible set of requirements.

Example: for a college bus-tracking app, the team interviews students and drivers, builds a screen mockup, finds that live GPS conflicts with battery limits, and negotiates a 30-second update interval.

In short, elicitation collects the wishes and analysis turns them into a consistent, buildable plan.

8 Why are requirements difficult to uncover? Explain. [Asked: Mar 2021, Feb 2025]

REPEATED

Requirements are difficult to uncover because customers often cannot state clearly what they need.

Users know their daily work but not how to express it as software features.

Different stakeholders give conflicting requirements that must be reconciled.

Requirements keep changing as customers see early versions and get new ideas.

Some requirements stay hidden because users assume certain things are obvious and never mention them.

Technical limits may make some wishes impossible, which only emerges during analysis.

The problem grows in large systems where hundreds of stakeholders are involved.

Skilled analysts use prototypes, observation and careful questioning to dig out the real needs.

Example: a shop owner says make billing fast but cannot define fast; the analyst discovers it means under 10 seconds per bill.

In short, elicitation is hard because human needs are vague, conflicting and moving, and the analyst must dig them out carefully.

9 Explain the requirements engineering processes.

IMPORTANT

Requirements engineering has four main processes that run in a loop.

Elicitation and analysis: gather the needs from stakeholders and understand them.

Specification: write the needs down clearly in the requirements document.

Validation: check that the written requirements are correct, complete and realistic.

Management: control changes to requirements as the project continues.

They are not done once; they repeat until everyone agrees and requirements are stable.

For example, a customer may add a new feature mid-project, which goes through all four steps again.

Skipping any of these steps leads to missing or wrong requirements and project failure.

10 Explain the techniques of requirements elicitation.

IMPORTANT

Interviews: meet stakeholders one by one and ask open questions about their needs.

Questionnaires: send written questions to collect answers from many users quickly and cheaply.

Observation: watch users doing their present work to find needs they forgot to mention.

Workshops: bring all stakeholders into one room to discuss and agree fast.

Document study: read existing forms, reports and manuals to understand current processes.

Each technique suits a different situation, so teams usually combine two or three of them.

For example, a hospital system team may interview doctors, observe nurses and hold a workshop with the admin staff.

Using only one technique often misses hidden needs, so mixing them gives complete requirements.

11 What is requirements analysis? Why is it needed?

IMPORTANT

Requirements analysis studies the gathered requirements to find conflicts, gaps and impossible demands.

Different stakeholders often ask for opposite things without realising it.

Example: one user wants instant reports while another wants heavy security checks on every report, and analysis resolves this conflict.

It also finds missing details, like what should happen when a payment fails.

Impossible demands are removed early, like a mobile app working with zero internet forever.

Tools like data flow diagrams and use case diagrams help model and check the requirements.

Without analysis, developers start coding on confused and conflicting requirements.

So analysis turns a raw pile of wishes into a clean, workable set of requirements.

12 What is requirements validation? Explain its techniques.

IMPORTANT

Requirements validation checks that the requirements are correct, complete and realistic.

It asks two questions: did we capture what the customer truly wants, and can we actually build it.

Technique one is reviews: stakeholders read the document together and point out errors.

Technique two is prototyping: build a quick demo so the customer can see and confirm.

Technique three is test-case writing: writing tests early exposes unclear or untestable requirements.

Catching an error during validation is far cheaper than catching it after coding is done.

For example, a review may reveal that a report the customer needs was never written down.

Validation is the quality gate that stops wrong requirements from entering design.

13 What is requirements management?

IMPORTANT

Requirements management controls changes to requirements during the project.

Requirements always change because customers understand their needs better over time.

Each requirement is tracked with a number, its version and its current status.

Every change request is analysed for cost, time and impact before it is approved.

Approved changes are updated in the document so everyone works on the latest version.

For example, adding fingerprint login mid-project needs approval and a schedule change.

Without management, uncontrolled changes cause confusion, delays and budget overruns.

Good management keeps requirements stable enough to build while still allowing useful changes.

14 What is negotiation in requirements engineering?

IMPORTANT

Negotiation is the process of resolving conflicts between different stakeholders' requirements.

Different users often want opposite things: managers want low cost while users want many features.

The analyst brings the parties together and helps them agree on priorities.

Less important requirements may be dropped or moved to a later release.

Win-win solutions are found where each side gives up something small.

Technical limits are explained honestly so stakeholders adjust their expectations.

The agreed decisions are recorded in the requirements document so nobody reopens settled points.

Example: students want bus tracking every 5 seconds but the transport office worries about data cost; they negotiate a 30-second interval.

In short, negotiation turns conflicting wishes into one agreed plan everyone can live with.

15 What is Facilitated Application Specification Technique (FAST)?

IMPORTANT

FAST is a team-based technique for gathering requirements in a guided joint session.

Customers and developers meet together instead of working through long separate interviews.

A neutral facilitator controls the meeting and keeps it focused on decisions.

Before the meeting, each participant prepares lists of what the system should do, what is already known, and what is unclear.

In the session the group discusses each list, removes duplicates and resolves conflicts on the spot.

A combined draft specification is prepared during or immediately after the meeting.

FAST works well when stakeholders are available and decisions need speed.

Example: for a canteen billing system, the owner, cashiers and developers sit together for a day and finalise the billing rules in one session.

In short, FAST speeds up elicitation by replacing slow one-by-one interviews with one focused team workshop.

TOPIC 4

4

Risk Concepts and Strategies

Risk is any uncertain event that can harm the project.

Reactive strategy: deal with problems only after they happen; also called firefighting.

Proactive strategy: identify risks early and plan how to avoid or reduce them.

Proactive is always better because fixing late costs much more.

QUESTION BANK

16 **What is software risk? Explain reactive vs proactive strategies.**

IMPORTANT

Software risk is any uncertain event that can harm the project if it happens.

It can hurt the schedule, the cost, the quality or even the success of the product.

A reactive strategy deals with problems only after they happen, which is also called firefighting.

The team waits for trouble and then rushes to fix it, which is costly and stressful.

A proactive strategy identifies risks early and plans how to avoid or reduce them.

The team prepares backup plans before anything goes wrong.

Proactive is always better because fixing a problem late costs much more than preventing it.

For example, training two people on a critical module is proactive, while struggling after one quits is reactive.

17 **Compare reactive and proactive risk strategies with an example.**

IMPORTANT

Reactive strategy has no risk planning; the team panics when trouble arrives.

Problems cause sudden delays, overtime work and poor quality fixes.

Proactive strategy lists risks early and keeps backup plans ready.

When a risk occurs, the team calmly follows the prepared plan.

Example: if a key developer may quit, proactive means training a second developer now.

The reactive version is hiring urgently after the developer has already quit, delaying the project.

Proactive teams stay on schedule while reactive teams keep firefighting.

So successful projects always choose the proactive strategy.

TOPIC 5

5

Types of Software Risks

Project risks: threaten the schedule or budget, like staff leaving.

Technical risks: threaten quality, like new untested technology.

Business risks: threaten the product success, like no market demand.

Knowing the type helps choose the right response.

QUESTION BANK

18 **Explain the types of software risks with examples.**

IMPORTANT

Project risks threaten the schedule or the budget of the project.

Example: a key developer resigns, or the hardware delivery is delayed by a month.

Technical risks threaten the quality or performance of the software.

Example: the chosen database cannot handle the expected number of users.

Business risks threaten the success or viability of the product in the market.

Example: customers do not want the product, or a competitor releases a better one first.

Knowing the type of risk helps the team choose the right response.

A project risk needs a backup plan for people, while a technical risk needs testing or a new design.

19 **What are known, predictable and unpredictable risks?**

IMPORTANT

Known risks can be listed from experience before the project starts.

Example: delay in hardware delivery or unclear customer requirements.

Predictable risks are estimated from data of past projects.

Example: the staff turnover rate in the company tells us some developers may leave.

Unpredictable risks are surprises that no one could see coming.

Example: a sudden new government law changes the tax rules the software must follow.

Known and predictable risks can be planned for in advance.

Unpredictable risks need a general buffer of time and budget to survive them.

TOPIC 6

6

Risk Identification, Projection and Refinement

Risk identification: list all possible risks for the project.

Risk projection: estimate each risk probability and its impact.

Risk refinement: break big risks into smaller detailed ones.

These steps turn vague worries into a managed risk list.

QUESTION BANK

20 **Explain risk identification, projection and refinement.**

IMPORTANT

Risk identification is listing all possible risks for the project.

It is done by brainstorming with the team and using checklists and past experience.

Risk projection estimates each risk in two ways: how likely it is and how much damage it causes.

A risk with high probability and high impact becomes the top priority.

Risk refinement breaks a big vague risk into smaller detailed ones.

For example, staff problems is refined into no training, poor communication and low motivation.

Detailed risks are easier to plan for than one big vague worry.

These three steps turn vague worries into a managed risk list the team can act on.

21 **What is a risk table? How is it used?**

IMPORTANT

A risk table is a list of all identified risks with their probability and impact level.

Each row has the risk name, chance of happening and damage if it happens.

Risks are sorted so the team handles the biggest ones first.

Example: an 80 percent chance of a 2-week delay ranks above a 10 percent chance of a 1-day delay.

The table is reviewed in every project meeting to check if risks are growing or shrinking.

New risks found during the project are added to the table.

It keeps risk handling organised instead of depending on memory.

Managers use it to decide where to spend time and money on risk control.

TOPIC 7

7

RMMM and RMMM Plan

RMMM stands for Risk Mitigation, Monitoring and Management.

Mitigation: steps to avoid the risk or reduce its damage.

Monitoring: watch warning signs that the risk is coming.

Management: backup plan to execute if the risk actually happens.

The RMMM plan documents all this for every major risk.

QUESTION BANK

22 **What is RMMM? Explain its four parts.**

IMPORTANT

RMMM stands for Risk Mitigation, Monitoring and Management.

It is the plan for handling each major risk in the project.

Mitigation: steps taken to avoid the risk or reduce its damage before it happens.

Monitoring: watching for warning signs that the risk is coming closer.

Management: the backup plan executed if the risk actually happens.

Together they cover the risk before, during and after it occurs.

Every major risk in the risk table gets its own RMMM plan.

RMMM turns risk from a fear into a managed, documented process.

23 **Prepare an RMMM plan for the risk of losing a key developer.**

IMPORTANT

Risk: the key developer working on the payment module may leave mid-project.

Mitigation: train a second developer on the critical modules right now so knowledge is shared.

Also document the module design and code clearly so a newcomer can understand it fast.

Monitoring: watch for warning signs like falling motivation, frequent leave or job interviews.

Hold regular one-to-one talks to catch dissatisfaction early.

Management: if the developer leaves, start the replacement hiring process within one week.

The trained second developer takes over temporarily so work does not stop.

This plan keeps the project safe even if the worst happens.

24 **Distinguish between risk mitigation and risk management.**

IMPORTANT

Mitigation is proactive: it acts before the risk happens to avoid or soften it.

Management is the backup action: it acts after the risk happens to recover.

Example: taking daily data backups is mitigation because it reduces damage.

Restoring the system from backup after a crash is management because it fixes the damage.

Mitigation reduces the chance or impact; management reduces the loss after impact.

Both are written in the RMMM plan for every major risk.

A project with only mitigation has no backup, and one with only management suffers full damage first.

So both are needed for complete risk control.

25 **What are the contents of an RMMM plan?**

IMPORTANT

An RMMM plan documents how each major risk will be handled.

It lists each major risk with its mitigation steps to avoid or reduce it.

It lists the monitoring indicators, the warning signs to watch for each risk.

It lists the management actions, the backup plan to run if the risk occurs.

It names who is responsible for watching and handling each risk.

It also notes the cost and time needed for each mitigation and backup action.

The plan is reviewed regularly because new risks appear during the project.

A complete RMMM plan means no major risk is left to chance.

TOPIC 8

8

Software Pricing

Software pricing decides how much to charge the customer.

Pricing can be based on cost plus profit, or on the value to the customer.

Wrong pricing loses money or loses the contract to competitors.

Pricing must consider effort, risk, hardware and maintenance costs.

26 Explain software pricing and its approaches.

IMPORTANT

Software pricing decides how much to charge the customer for building the software.

Cost-based pricing estimates the total cost and then adds a profit margin on top.

Value-based pricing charges based on the benefit the customer gets from the software.

Example: a billing system that saves the customer 10 lakhs a year can be priced higher than its build cost.

Market-based pricing also looks at what competitors charge for similar work.

Pricing must consider effort, risk, hardware and future maintenance costs.

The final price is quoted in the contract before the project starts.

Good pricing keeps the company profitable while winning the contract.

27 What are the risks of wrong software pricing?

IMPORTANT

Underpricing means the company loses money even if the project succeeds.

Overpricing means the customer rejects the quote and the contract goes to competitors.

Ignoring maintenance cost causes losses after delivery when bugs must be fixed free.

Ignoring risk means unexpected problems eat up all the profit.

For example, a fixed-price quote without a risk buffer can turn into a loss if requirements change.

Wrong pricing can also damage the company reputation if quality is cut to save cost.

So pricing mistakes hurt money, customers and reputation together.

Good pricing balances cost, value, risk and competition carefully.

TOPIC 9

Plan-Driven Development and Project Scheduling

9

Plan-driven development makes a complete plan before building.

Project scheduling breaks work into tasks, assigns people and sets deadlines.

Tools like Gantt charts show tasks on a timeline; PERT charts show task dependencies.

The critical path is the longest task chain; any delay there delays the project.

28 What is plan-driven development? [Asked: Model paper]

REPEATED

Plan-driven development creates the complete plan, schedule and documents before coding starts.

All requirements are fixed first, then design, then implementation in order.

It suits projects with stable requirements and strict deadlines.

It gives the customer clear dates, costs and documents at every stage.

The waterfall model is the classic example of plan-driven development.

Its weakness is that changing requirements mid-way is costly and difficult.

It works best for critical systems like banking or medical software needing full documentation.

So plan-driven means plan everything first, then build exactly as planned.

29 Describe project planning in software engineering. [Asked: Model paper]

REPEATED

Project planning is the activity of deciding how a software project will be executed before the work starts.

It sets the project scope: what will be built and what will not.

It breaks the work into tasks and assigns them to team members with deadlines.

It estimates the effort, cost and time needed, using techniques like COCOMO or expert judgement.

It identifies risks and prepares responses for them in advance.

It creates a schedule, often shown as a Gantt chart, so progress can be tracked week by week.

It plans quality checks, reviews and testing at the right points in the timeline.

It also plans the team structure, tools and communication methods.

Example: before building a school fee portal, the plan lists modules, assigns two developers, estimates three months, and schedules testing in the final two weeks.

In short, project planning turns a vague idea into a scheduled, costed and risk-aware path to delivery.

30 What is project scheduling? Explain with Gantt chart.

IMPORTANT

Project scheduling divides the project into tasks with durations and deadlines.

Each task is assigned people, resources and a start and end date.

Tasks are ordered by dependencies: some tasks must finish before others begin.

A Gantt chart shows this schedule as horizontal bars on a timeline.

Each bar shows a task, its length shows duration, and its position shows timing.

The manager tracks progress by comparing actual work against the chart.

Delays are visible at once because bars shift to the right.

Good scheduling keeps everyone aware of what to do and when to finish it.

31 What is the critical path in project scheduling?

IMPORTANT

The critical path is the longest sequence of dependent tasks in the project.

It decides the shortest possible time in which the project can finish.

If any task on this path is delayed, the whole project is delayed.

Tasks not on the critical path have slack, meaning they can be delayed a little without harm.

Managers watch critical path tasks most closely and give them the best people.

PERT charts help find the critical path by showing task dependencies.

For example, in a college portal project, database design may lie on the critical path.

Protecting the critical path is the key to delivering on time.

32 What is the purpose of a feasibility study?

IMPORTANT

A feasibility study checks whether a proposed software project is worth doing before money is spent.

Technical feasibility asks whether the team has the technology and skill to build it.

Economic feasibility asks whether the benefits will be greater than the cost.

Operational feasibility asks whether the organisation can actually use and maintain the system.

Legal feasibility checks that the project breaks no laws, licences or contracts.

Schedule feasibility checks whether it can be finished in the available time.

If any dimension fails badly, the project should be reshaped or cancelled early.

Example: before building a drone-delivery app, the study might find that local aviation rules

make it legally infeasible.

In short, a feasibility study answers should we build this at all before the team commits.

33 **Distinguish between deliverables and milestones.**

IMPORTANT

Deliverables are the actual products handed over; milestones are the checkpoints that mark progress.

A deliverable is something the customer receives, like a tested module, a design document or a user manual.

A milestone is a point in time when a major result is achieved, like completing the design phase.

Deliverables are judged by quality; milestones are judged by date.

Every milestone usually produces one or more deliverables as proof.

Missing a deliverable means the work is incomplete; missing a milestone means the schedule has slipped.

Both are fixed in the project plan so progress is visible and measurable.

Example: a tested login module is a deliverable, while login module completed by 10 March is a milestone.

In short, deliverables are what you give, milestones are when you prove progress.

TOPIC 10

10

Agile Planning

Agile planning plans only as far as needed, then adapts.

Release planning decides which features go into the next release.

Iteration planning picks the work for the next sprint or two weeks.

Plans are updated every iteration based on feedback and velocity.

34 Explain agile planning.

IMPORTANT

Agile planning plans only as far as needed and then adapts as work continues.

Instead of one big plan, planning happens at three levels.

Release planning decides which features go into the next release.

Iteration planning picks the work for the next sprint, usually two weeks.

Daily planning happens in the standup meeting where the team adjusts the day's work.

Plans are updated every iteration based on feedback and velocity.

Velocity is the amount of work finished per sprint, and it guides how much to promise next time.

So agile planning stays flexible and always reflects the latest reality.

35 Compare traditional and agile project planning.

IMPORTANT

Traditional planning makes one big plan at the start and changing it is hard.

Agile planning makes small plans every iteration and changing them is easy.

Traditional plans are detailed and document-heavy; agile plans are light and visual.

Traditional suits fixed scope projects with stable requirements.

Agile suits evolving scope where customer feedback keeps changing priorities.

In traditional planning, delays are found late; in agile, velocity shows problems every sprint.

For example, a government tender project uses traditional planning while a startup app uses agile.

The choice depends on how stable the requirements are.

TOPIC 11

Estimation Techniques

11

Estimation predicts how much effort, time and cost a project needs.

Expert judgement: experienced people guess based on past projects.

COCOMO: a formula model using project size to estimate effort.

Story points: agile teams estimate task size relatively, not in hours.

Good estimation needs historical data from similar projects.

36 **Write a short note on the COCOMO model. [Asked: May 2019, Oct 2021, Aug 2022, Feb 2025]**

SURE MARK

COCOMO stands for Constructive Cost Model, created by Barry Boehm.

It estimates effort, time and cost from the estimated size of the software in lines of code.

It uses mathematical formulas tuned with data from many past projects.

There are three levels: basic for quick estimates, intermediate with cost factors, and detailed for each phase.

It also classifies projects as organic, semi-detached or embedded based on complexity.

COCOMO suits large plan-driven projects better than small agile ones.

Its accuracy depends on how correctly the size is estimated at the start.

It remains one of the most famous algorithmic estimation models in software engineering.

37 **What is estimation? Explain the estimation techniques. [Asked: Model paper]**

REPEATED

Estimation predicts how much effort, time and cost a project needs before building starts.

Expert judgement asks experienced people to guess based on similar past projects.

Analogy compares the new project with a completed one and adjusts the numbers.

The COCOMO model uses formulas based on the estimated size of the software.

Story points let agile teams estimate task size relatively instead of in hours.

Good estimation needs historical data from similar projects.

Bad estimation causes missed deadlines, budget overruns and unhappy customers.

So estimation is a skill every project manager must master.

38 **Explain function point analysis. [Asked: Oct 2021, Aug 2022]**

REPEATED

Function point analysis is an estimation technique that measures software size by its functionality, not by lines of code.

It counts five functional units: external inputs, external outputs, external inquiries, internal logical files and external interface files.

Each unit is rated as simple, average or complex and given a weight.

The weighted sum gives the unadjusted function point count.

This count is then adjusted using fourteen general system characteristics, such as performance

and reusability needs.

The final function point total is converted into effort and cost using the organisation's past project data.

Because it needs no code, it can estimate very early, right after requirements are known.

Example: a student record system with 10 inputs, 8 outputs and 3 files might total 120 function points, which past projects show needs about 4 person-months.

In short, function points estimate size from what the software does, making early estimates possible.

39 What are story points in agile estimation?

IMPORTANT

Story points measure task size relatively instead of in hours.

The team compares a new task with completed ones: this task is twice as big as that one.

A simple 1, 2, 3, 5, 8 scale is commonly used, like the Fibonacci series.

Points cover effort, complexity and uncertainty together, not just time.

After a few sprints, the team knows its velocity: the points finished per sprint.

Velocity is used to plan how many points the team can take in future sprints.

For example, a team with velocity 20 will not promise 30 points next sprint.

Story points make agile planning realistic and free from false hour promises.